

1. Task Overview

Task Title: Network Security Monitoring using Suricata IDS/IPS

Task Context: This report documents the installation and configuration of Suricata on Ubuntu Server 24.04, covering rule management, live IDS alert capture, custom rule creation, IPS traffic blocking, and EVE JSON log analysis.

2. Objective

To install and configure Suricata as a Network Intrusion Detection and Prevention System, validate detection using Emerging Threats rules, write custom rules, demonstrate active traffic blocking in IPS mode, and analyze structured log output.

Task S1 Install Suricata and Load Rules

Suricata was installed on Ubuntu Server 24.04 using apt. Build info was verified to confirm AF_PACKET and NFQueue support, which are required for live capture and IPS mode respectively. The suricata-update tool was installed separately to download the Emerging Threats Open ruleset.

```
tasbiha101@fjerda:~$ suricata --build-info | grep -E "NFQ|AF_PACKET|Version"
Features: NFQ PCAP_SET_BUFF AF_PACKET HAVE_PACKET_FANOUT LIBCAP_NG LIBNET1.1 HAVE_HTTP_URI_NORMALIZE_HOOK PCRE_JIT HAVE_N
JIT HAVE_LIBJANSSON TLS TLS_C11 MAGIC RUST
AF_PACKET support: yes
NFQueue support: yes
tasbiha101@fjerda:~$
```

After running suricata-update, the configuration was tested in dry-run mode (-T flag). The output confirmed successful loading with no errors.

```
tasbiha101@fjerda:~$ sudo suricata -T -c /etc/suricata/suricata.yaml
[sudo] password for tasbiha101:
i: suricata: This is Suricata version 7.0.3 RELEASE running in SYSTEM mode

i: suricata: Configuration provided was successfully loaded. Exiting.
tasbiha101@fjerda:~$
```

Total rules loaded: 65,741 | Rules enabled: 49,068 | Suricata version: 7.0.3

Task S2 Live IDS Mode Alert Capture

Suricata was started in IDS mode on interface enp0s3. A test request was made to testmynids.org using curl, which is a site specifically designed to trigger IDS rules by returning a response containing uid=0(root) — simulating a post-exploitation server response.

Alert Name	GPL ATTACK_RESPONSE id check returned root
Alert Time	04/30/2026 - 09:37:56
Host	Ubuntu Server (10.0.2.5)
Source IP	43.204.106.89 (testmynids.org)
Rule ID	2100498
Classification	Potentially Bad Traffic
Initial Severity	Priority 2 (High)

```

ZelliJ (diligent-hill)
Pane #1
tasbiha101@fjenda:~$ sudo tail -f /var/log/suricata/fast.log
[sudo] password for tasbiha101:
04/30/2026-09:37:56.118979  [**] [1:2100498:7] GPL ATTACK_RESPONSE id check returned root [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP}
43.204.106.89:80 -> 10.0.2.5:43814

```

Additionally, Suricata automatically detected Kali Linux joining the network via a DHCP broadcast packet (Rule 2022973 — ET INFO Possible Kali Linux hostname in DHCP Request Packet, Priority 1), demonstrating that the sensor was actively monitoring all NatNetwork traffic.

3. Triage Questions and Answers

Question	Analyst Note
What happened?	A test request to testmynids.org triggered a Suricata rule detecting a server response containing uid=0(root), simulating a post-exploitation response.
Where did it happen?	Ubuntu Server (10.0.2.5) acting as the IDS sensor on the NatNetwork lab environment.
Who was involved?	Ubuntu Server initiated the curl request; testmynids.org (43.204.106.89) returned the triggering response.
Is the behavior expected?	Yes, in this context. The site is designed specifically to test IDS functionality. In a real environment this would be a high-priority alert.
What is missing?	User context and process information on what initiated the request. In a real incident, endpoint telemetry would be checked.

Task S3 Custom Rule: YouTube Detection

The connection failed immediately as expected. Suricata processed 79 packets, accepted 55 and dropped 24, confirming active packet interception was working.

Note: The initial block manifested as Network is unreachable on the IPv6 address. IPv4 attempts also failed to complete, confirming the IPS rule was enforced. After the test, iptables rules were flushed using `sudo iptables -F` to restore normal traffic flow.

```
tasbiha1010fjerda:~$ sudo kill suricata
tasbiha1010fjerda:~$ sudo iptables -D INPUT -j NFQUEUE --queue-num 0
tasbiha1010fjerda:~$ sudo iptables -D OUTPUT -j NFQUEUE --queue-num 0
iptables: Bad rule (does a matching rule exist in that chain?).
tasbiha1010fjerda:~$ sudo iptables -D OUTPUT -j DROP
iptables: Bad rule (does a matching rule exist in that chain?).
tasbiha1010fjerda:~$ sudo iptables -L | head -5
Chain INPUT (policy ACCEPT)
target     prot opt source                destination           NFQUEUE num 0
NFQUEUE   all  --  anywhere              anywhere              NFQUEUE num 0

Chain FORWARD (policy ACCEPT)
tasbiha1010fjerda:~$
```

Task S5 — EVE JSON Log Parsing

The EVE JSON log at `/var/log/suricata/eve.json` was parsed using `jq` to extract and analyze alert signatures. This log contains structured JSON for every event type including alerts, DNS, HTTP, TLS, and flows.

```
tasbiha1010fjerda:~$ jq 'select(.event_type=="alert") | .alert.signature' /var/log/suricata/eve.json
"GPL ATTACK_RESPONSE id check returned root"
"ET INFO Possible Kali Linux hostname in DHCP Request Packet"
"GPL ATTACK_RESPONSE id check returned root"
"Youtube Access Detected"
"Youtube Access Detected"
tasbiha1010fjerda:~$ _
```

```
tasbiha1010fjerda:~$ jq -r 'select(.event_type=="alert") | .alert.signature' /var/log/suricata/eve.json | sort | uniq -c | sort -rn
2 Youtube Access Detected
2 GPL ATTACK_RESPONSE id check returned root
1 ET INFO Possible Kali Linux hostname in DHCP Request Packet
tasbiha1010fjerda:~$
```

The most frequently triggered rule was the custom YouTube detection rule and the GPL ATTACK_RESPONSE rule — both fired twice due to bidirectional traffic capture.

6. Analyst Decision

All alerts generated during this task were the result of deliberate test actions and are classified as True Positives triggered in a controlled lab environment. The Suricata deployment successfully detected application-layer content in HTTP traffic, identified a known pentesting OS joining the network, and actively blocked outbound connections in IPS mode. No false positives were observed. The custom rule confirmed that Suricata can be extended beyond its default ruleset for environment-specific detection needs.

7. Recommended Next Actions

5. Integrate Suricata EVE JSON output with Wazuh SIEM for centralized alert correlation
6. Add Zeek alongside Suricata for full session logging and forensic context
7. Enable promiscuous mode permanently via netplan for broader network visibility
8. Tune rule thresholds to reduce noise from low-severity informational rules
9. Schedule daily `suricata-update` via cron to keep threat signatures current

8. Short Analyst Summary

In this task I set up Suricata as a network IDS/IPS on Ubuntu Server. The tool successfully loaded 65,741 Emerging Threats rules and detected live traffic including a test attack response and a Kali Linux host joining the network. I wrote a custom rule to detect YouTube traffic which fired correctly in both directions. In IPS mode, Facebook traffic was actively dropped using NFQUEUE. EVE JSON logs were parsed with jq to identify the most frequent alerts. The main limitation observed was that VirtualBox NAT Network does not forward cross-VM traffic through the sensor by default..